

Contents

Tutorial 1		1	4	Improving Naïve Search	1
1	Subsequence	1	5	Knuth–Morris–Pratt Example	1
2	Longest Common Subsequence	1	6	Longest Palindromic Subsequence	1
3	Naïve Search	1			

For the purpose of these exercises we assume that array indices start at 0, i.e. an array A of n integers has indices $0, \dots, n-1$. A string is considered to be an array of characters.

1. Subsequence Describe an efficient algorithm to check whether a string $\mathbf{a} = (a_1, \dots, a_n)$ contains $\mathbf{b} = (b_1, \dots, b_n)$ as an ordered (but not necessarily continuous) subsequence string. What if we want to count the number of such occurrences?

2. Longest Common Subsequence If a sequence of characters $\mathbf{c} = (c_1, \dots, c_k)$ is a subsequence of both $\mathbf{a} = (a_1, \dots, a_n)$ and $\mathbf{b} = (b_1, \dots, b_m)$, we call $\mathbf{c}$ *common* to $\mathbf{a}$ and $\mathbf{b}$. Note that subsequence need not be continuous.

Describe an efficient algorithm for finding a longest common subsequence of two given sequences of characters.

3. Longest Palindromic Subsequence Denote by α^{-1} the reversed string α so that $\alpha^{-1}[i] = \alpha[-i]$ for each index i . We say that a string is a *palindrome* if $\alpha^{-1} = \alpha$.

Describe a dynamic algorithm that finds a longest palindromic subsequence in a given sequence of characters $a = (a_1, \dots, a_n)$.

4. Naïve Search The naïve string matching algorithm prints starting indices of all occurrences of a pattern string P within text string T and can be formulated as follows.

Algorithm: Naïve string matching algorithm

Input: Pattern string $P[0 \dots m-1]$, text string $T[0 \dots n-1]$.
Output: Prints starting positions of pattern P within text T .

```

for  $i \leftarrow 0$  to  $n - m$  do
     $k \leftarrow 0$ 
    while  $k < m$  and  $P[k] = T[i + k]$  do
         $k \leftarrow k + 1$ 
    if  $k = m$  then
        print  $i$ 

```

Show that, for every m and n with $m < n$, there is a text T of length n and a pattern P of length m such that the naïve string matching performs $\Omega(nm)$ steps (character to character comparisons). Even if P has no occurrence in T .

5. Improving Naïve Search Assuming that the pattern string P contains no repeated character, modify the naïve string matching algorithm to work in linear time.

What if no character occurs more than two times?

A refresher of definitions:

- The KMP (Knuth-Morris-Pratt) automaton is at state α after processing a string σ iff α is the longest suffix of σ which is also a prefix of the needle ι .
 - When the KMP automaton is at state α and does not match the next character, it takes a back edge $Z(\alpha)$ which goes to the longest proper suffix of α which is also a prefix of ι .
-

6. Knuth–Morris–Pratt Example Construct the search automata for the words *kokos* and *ananas*.